



BRAINWARE UNIVERSITY

BRAINWARE UNIVERSITY

ML PROJECT-BASED LEARNING REPORT (WEEK 2)

1. Basic Information

Title of the Project:	Electricity Bill Calculator & ATM Transaction Simulator (Program 1 & 2)
Name(s) of Student(s):	Soumyadwip Das (BWU/BTS/25/169) Ankita Paul (BWU/BTS/25/180) Sonia Naskar (BWU/BTS/25/195) Joyeta Mondal (BWU/BTS/25/214) Sayantan Bharati (BWU/BTS/25/503)
Programme & Semester:	B.Tech CSE & 3rd Semester
Course Name & Code:	Python Programming Lab (BES09013)
Name of the Guide/Supervisor:	MR. PRANAB GHARAI & MR. INDRANIL DAS



2. Introduction and Background

Week 2 focused on developing two console-based Python applications: an **Electricity Bill Calculator** and an **ATM Transaction Simulator**. The programs apply core Python concepts such as conditional logic, functions, dictionaries, and menu-driven control flow, using in-memory data storage and a consistent boxed terminal interface.

3. Objectives of the Project

3a. Objectives - Program 1: Electricity Bill Calculator

- Develop a menu-driven electricity bill calculator.
- Implement slab-wise billing logic.
- Apply discount and fixed service charge rules.
- Store user records using dictionaries.
- Validate user input.

3b. Objectives - Program 2: ATM Transaction Simulator

- Develop a console-based ATM simulator.
- Implement user authentication.
- Support balance enquiry, withdrawal, deposit, and PIN change.
- Store account details using dictionaries.
- Test the program with valid and invalid inputs.

4. Problem Statement / Driving Question

4a. Program 1: Electricity Bill Calculator

How can a terminal application accurately calculate electricity bills using slab-wise tariffs, discounts, and fixed charges while managing multiple user records?

4b. Program 2: ATM Transaction Simulator

How can common ATM operations be simulated securely in a console application while handling invalid credentials and transactions?



5. Methodology

5a. Methodology - Program 1: Electricity Bill Calculator

1. Program Design

- Developed a menu-driven interface with reusable box-drawing functions and an ASCII-art banner.

2. Billing Logic Implementation

- Implemented `calc()` to calculate slab-wise electricity charges, apply discounts, and add a fixed service charge.

3. Data Handling & Validation

- Implemented Stored user records in an in-memory dictionary and validated unit input using exception handling.

5b. Methodology - Program 2: ATM Transaction Simulator

1. Program Structure

- Created a boxed menu interface using reusable helper functions and an ASCII-art banner.

2. Account Data Modelling

- Stored account details in a Python dictionary containing card number, PIN, and balance.

3. Core Transaction Logic

- Implemented authentication, balance enquiry, withdrawal, deposit, and PIN change with necessary validation.

4. Menu-Driven Control Flow

- Built a continuous menu system and tested various valid and invalid transaction scenarios.

6. Expected Outcomes / Deliverables

6a. Program 1: Electricity Bill Calculator

- A functional slab-wise electricity bill calculator.
- Accurate discount and fixed-charge calculation.
- Menu-driven interface with user record management.
- Scope for future enhancements.

6b. Program 2: ATM Transaction Simulator

- A functional ATM simulator with authentication and transactions.
- Input validation for users and transactions.
- User-friendly boxed terminal interface.
- Scope for future enhancements such as persistent storage and account security.



7. Core Logic

7a. Core Logic - Program 1: Electricity Bill Calculator

The `calc()` function is the core of the program. It determines the bill using slab-wise rates, applies a threshold-based discount, and adds a fixed charge:

```
def calc(unit):
    bill = 0
    discount = 0
    charge = FIXED_CHARGE

    # Slab-wise tariff: rate increases as consumption rises
    if unit <= 100:
        bill = unit * 1.5
    elif unit <= 200:
        bill = (100 * 1.5) + ((unit - 100) * 2.5)
    elif unit <= 300:
        bill = (100 * 1.5) + (100 * 2.5) + ((unit - 200) * 4.0)
    else:
        bill = (100 * 1.5) + (100 * 2.5) + (100 * 4.0) + ((unit - 300) * 6.0)

    # 10% discount applies only above a Rs. 1000 threshold
    if bill > 1000:
        discount = DISCOUNT_RATE

    amount_after_discount = bill - (bill * discount) / 100
    final_bill = amount_after_discount + charge # fixed charge added last

    return final_bill
```

In short: the function accumulates cost slab by slab as units increase, applies a 10% discount only when the pre-charge bill exceeds Rs. 1000, and finally adds the fixed Rs. 50 service charge before returning the total.

7b. Core Logic - Program 2: ATM Transaction Simulator

The authentication and withdrawal routines form the operational core of the simulator. `authenticate()` gatekeeps every transaction by verifying the card number and PIN before any account data is accessed, while `withdraw()` enforces that a transaction cannot exceed the available balance.

```
def authenticate(card_no, pin):
    if card_no in ACCOUNTS and ACCOUNTS[card_no]["pin"] == pin:
        return True # credentials match
    return False # invalid card/PIN

def withdraw(card_no, amount):
    if amount <= ACCOUNTS[card_no]["balance"]:
        ACCOUNTS[card_no]["balance"] -= amount # deduct funds
        return True
    return False # insufficient balance
```

Note: `MAX_ATTEMPTS` is currently declared but not yet enforced in the login flow; wiring it into `authenticate()` to lock a card after repeated failed attempts is planned as a future enhancement.



8. Output Screenshot

8a. Program 1: Electricity Bill Calculator

```
ELECTRICITY
BILL
CALCULATOR

Terminal Electricity Bill Calculator

[1] Calculate Bill
[2] Display Bill Status
[3] Show All User Info
[4] Exit

> Enter option: 1
> Enter your name: ram
> Enter the unit amount: 500

BILL CALCULATED:

Final bill for 500.0 unit(s) is: 1850.0

[1] Calculate Bill
[2] Display Bill Status
[3] Show All User Info
[4] Exit

> Enter option: 
```





9. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Core programming language for both programs
Python Built-in Features	Functions, dictionaries, input/output, exception handling, and f-strings
Python Dictionaries	Store billing and ATM account records
f-Strings	Format billing and transaction details
Functions & Control Flow	Implement billing, authentication, and transaction logic
Command-Line Interface	Run and test the menu-driven programs
Text Editor / IDE	Write and debug Python code

10. References

- Python Official Documentation
- Python Exception Handling Documentation
- Python String Formatting (f-Strings) Documentation
- Python Dictionaries Documentation
- GeeksforGeeks – Python Electricity Bill Calculator Tutorials
- Brainware University – Python Learning Course Guidelines



BRAINWARE UNIVERSITY

11. Signatures

Signature of Student(s):

Date of Submission:

Signature of Guide/Supervisor:
